Multi-Layer Perceptron, MNIST

In this notebook, we will train an MLP to classify images from the MNIST database hand-written digit database.

> 1. Load and visualize the data

The process will be broken down into the following steps: 2. Define a neural network 3. Train the model 4. Evaluate the performance of our trained model on a test dataset!

Before we begin, we have to import the necessary libraries for working with data and PyTorch.

In [2]:
```
# import libraries
import torch
import numpy as np
```

Load and Visualize the Data

Downloading may take a few moments, and you should see your progress as the data is loading. You may also choose to change the `batch_size` if you want to load more data at a time.

This cell will create DataLoaders for each of our datasets.

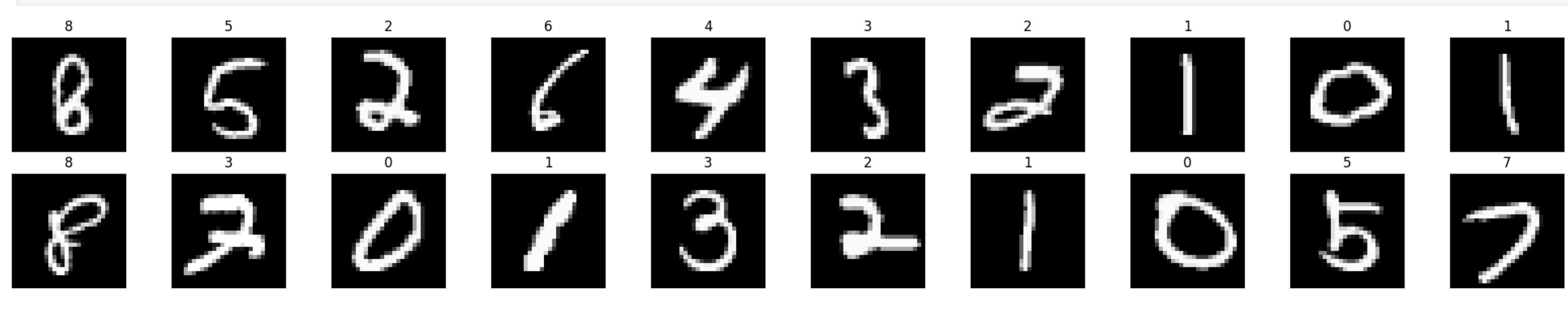
In [3]:
```
# The MNIST datasets are hosted on yann.lecun.com that has moved under CloudFlare protection
# Run this script to enable the datasets download
# Reference: https://github.com/pytorch/vision/issues/1938

from six.moves import urllib
opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
urllib.request.install_opener(opener)
```

In [28]:
```
from torchvision import datasets
import torchvision.transforms as transforms
from torch.utils.data.sampler import SubsetRandomSampler

# number of subprocesses to use for data loading
num_workers = 0
# how many samples per batch to load
batch_size = 20
# percentage of training set to use as validation
valid_size = 0.2

# convert data to torch.FloatTensor
transform = transforms.ToTensor()

# choose the training and test datasets
train_data = datasets.MNIST(root='data', train=True,
                                   download=True, transform=transform)
test_data = datasets.MNIST(root='data', train=False,
                                  download=True, transform=transform)

# obtain training indices that will be used for validation
num_train = len(train_data)
indices = list(range(num_train))
np.random.shuffle(indices)
split = int(np.floor(valid_size * num_train))
train_idx, valid_idx = indices[split:], indices[:split]

# define samplers for obtaining training and validation batches
train_sampler = SubsetRandomSampler(train_idx)
valid_sampler = SubsetRandomSampler(valid_idx)

# prepare data loaders
train_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size,
    sampler=train_sampler, num_workers=num_workers)
valid_loader = torch.utils.data.DataLoader(train_data, batch_size=batch_size,
    sampler=valid_sampler, num_workers=num_workers)
test_loader = torch.utils.data.DataLoader(test_data, batch_size=batch_size,
    num_workers=num_workers)
```

Visualize a Batch of Training Data

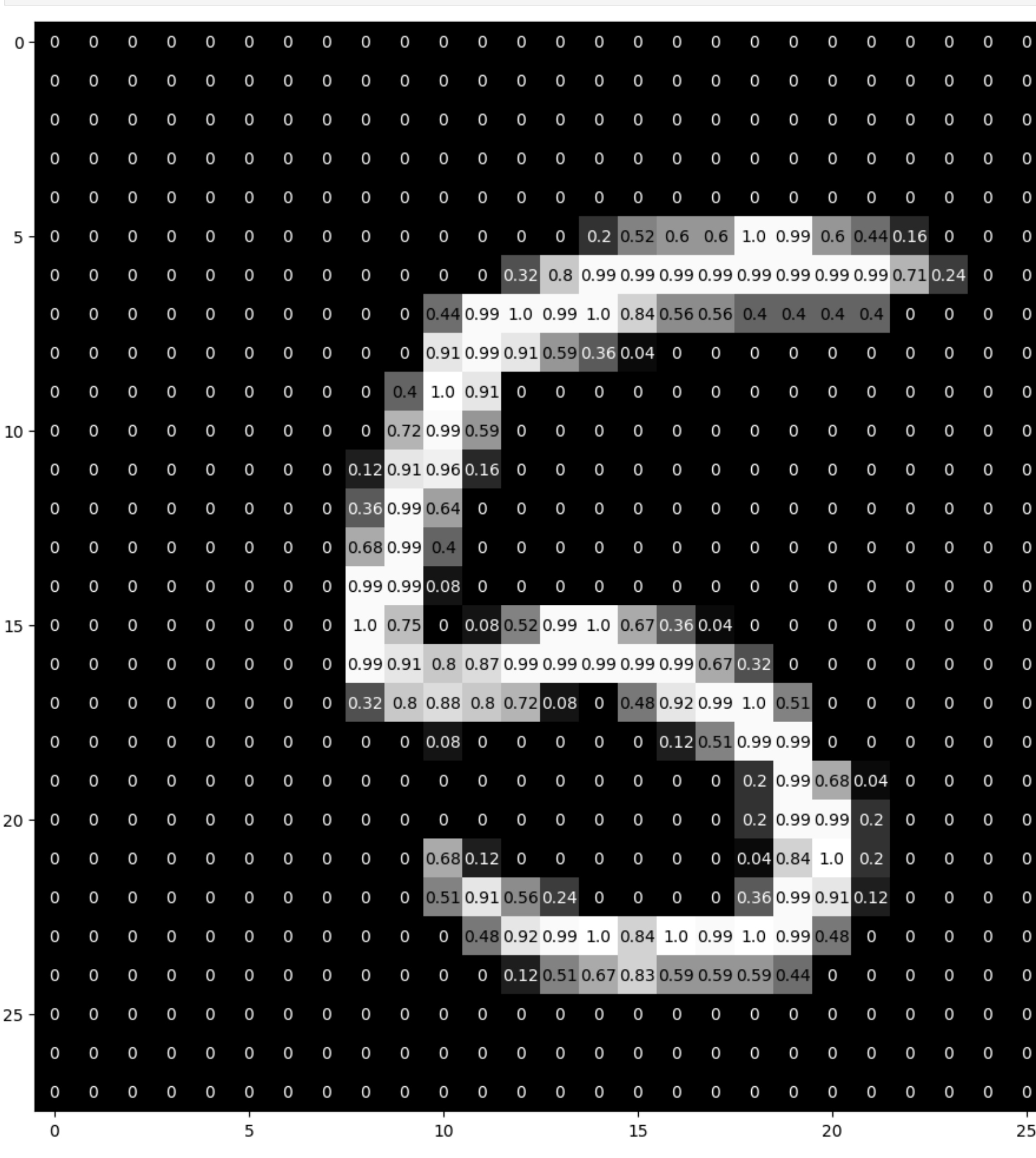
The first step in a classification task is to take a look at the data, make sure it is loaded in correctly, then make any initial observations about patterns in that data.

In [31]:
```
import matplotlib.pyplot as plt
%matplotlib inline

# obtain one batch of training images
dataiter = iter(train_loader)
images, labels = next(dataiter)
images = images.numpy()

# plot the images in the batch, along with the corresponding labels
fig = plt.figure(figsize=(25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 20//2, idx+1, xticks=[], yticks=[])
    ax.imshow(np.squeeze(images[idx]), cmap='gray')
    # print out the correct label for each image
    # .item() gets the value contained in a Tensor
    ax.set_title(str(labels[idx].item()))
```



View an Image in More Detail

In [32]:
```
img = np.squeeze(images[1])

fig = plt.figure(figsize = (12,12))
ax = fig.add_subplot(111)
ax.imshow(img, cmap='gray')
width, height = img.shape
thresh = img.max()/2.5
for x in range(width):
    for y in range(height):
        val = round(img[x][y],2) if img[x][y] !=0 else 0
        ax.annotate(str(val), xy=(y,x),
                    horizontalalignment='center',
                    verticalalignment='center',
                    color='white' if img[x][y]<thresh else 'black')
```



Define the Network Architecture

The architecture will be responsible for seeing as input a 784-dim Tensor of pixel values for each image, and producing a Tensor of length 10 (our number of classes) that indicates the class scores for an input image. This particular example uses two hidden layers and dropout to avoid overfitting.

In [33]:
```
import torch.nn as nn
import torch.nn.functional as F

# define the NN architecture
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        # number of hidden nodes in each layer (512)
        hidden_1 = 512
        hidden_2 = 512
        # linear layer (784 -> hidden_1)
        self.fc1 = nn.Linear(28 * 28, hidden_1)
        # linear layer (n_hidden -> hidden_2)
        self.fc2 = nn.Linear(hidden_1, hidden_2)
        # linear layer (n_hidden -> 10)
        self.fc3 = nn.Linear(hidden_2, 10)
        # dropout layer (p=0.2)
        # dropout prevents overfitting of data
        self.dropout = nn.Dropout(0.2)

    def forward(self, x):
        # flatten image input
        x = x.view(-1, 28 * 28)
        # add hidden layer, with relu activation function
        x = F.relu(self.fc1(x))
        # add dropout layer
        x = self.dropout(x)
        # add hidden layer, with relu activation function
        x = F.relu(self.fc2(x))
        # add dropout layer
        x = self.dropout(x)
        # add output layer
        x = self.fc3(x)
        return x

# initialize the NN
model = Net()
print(model)
```

```
Net(
  (fc1): Linear(in_features=784, out_features=512, bias=True)
  (fc2): Linear(in_features=512, out_features=512, bias=True)
  (fc3): Linear(in_features=512, out_features=10, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

Specify Loss Function and Optimizer

It's recommended that you use cross-entropy loss for classification. If you look at the documentation (linked above), you can see that PyTorch's cross entropy function applies a softmax funtion to the output layer *and* then calculates the log loss.

In [34]:
```
# specify loss function (categorical cross-entropy)
criterion = nn.CrossEntropyLoss()

# specify optimizer (stochastic gradient descent) and learning rate = 0.01
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

Train the Network

The steps for training/learning from a batch of data are described in the comments below:

1. Clear the gradients of all optimized variables
2. Forward pass: compute predicted outputs by passing inputs to the model
3. Calculate the loss
4. Backward pass: compute gradient of the loss with respect to model parameters
5. Perform a single optimization step (parameter update)
6. Update average training loss

The following loop trains for 30 epochs; feel free to change this number. For now, we suggest somewhere between 20-50 epochs. As you train, take a look at how the values for the training loss decrease over time. We want it to decrease while also avoiding overfitting the training data.

In [36]:
```
# number of epochs to train the model
n_epochs = 50

# initialize tracker for minimum validation loss
valid_loss_min = np.inf # set initial "min" to infinity

for epoch in range(n_epochs):
    # monitor training loss
    train_loss = 0.0
    valid_loss = 0.0

    ###################
    # train the model #
    ###################
    model.train() # prep model for training
    for data, target in train_loader:
        # clear the gradients of all optimized variables
        optimizer.zero_grad()
        # forward pass: compute predicted outputs by passing inputs to the model
        output = model(data)
        # calculate the loss
        loss = criterion(output, target)
        # backward pass: compute gradient of the loss with respect to model parameters
        loss.backward()
        # perform a single optimization step (parameter update)
        optimizer.step()
        # update running training loss
        train_loss += loss.item()

    ######################
    # validate the model #
    ######################
    model.eval() # prep model for evaluation
    for data, target in valid_loader:
        # forward pass: compute predicted outputs by passing inputs to the model
        output = model(data)
        # calculate the loss
        loss = criterion(output, target)
        # update running validation loss
        valid_loss += loss.item()

    # print training/validation statistics
    # calculate average loss over an epoch
    train_loss = train_loss/len(train_loader)
    valid_loss = valid_loss/len(valid_loader)

    print('Epoch: {} \tTraining Loss: {:.6f} \tValidation Loss: {:.6f}'.format(
        epoch+1,
        train_loss,
        valid_loss
        ))

    # save model if validation loss has decreased
    if valid_loss <= valid_loss_min:
        print('Validation loss decreased ({:.6f} --> {:.6f}).  Saving model ...'.format(
        valid_loss_min,
        valid_loss))
        torch.save(model.state_dict(), 'model.pt')
        valid_loss_min = valid_loss
```

```
Epoch: 1 	Training Loss: 0.944155 	Validation Loss: 0.378676
Validation loss decreased (inf --> 0.378676).  Saving model ...
Epoch: 2 	Training Loss: 0.358453 	Validation Loss: 0.291630
Validation loss decreased (0.378676 --> 0.291630).  Saving model ...
Epoch: 3 	Training Loss: 0.282207 	Validation Loss: 0.239186
Validation loss decreased (0.291630 --> 0.239186).  Saving model ...
Epoch: 4 	Training Loss: 0.231829 	Validation Loss: 0.202413
Validation loss decreased (0.239186 --> 0.202413).  Saving model ...
Epoch: 5 	Training Loss: 0.198319 	Validation Loss: 0.177197
Validation loss decreased (0.202413 --> 0.177197).  Saving model ...
Epoch: 6 	Training Loss: 0.170963 	Validation Loss: 0.158643
Validation loss decreased (0.177197 --> 0.158643).  Saving model ...
Epoch: 7 	Training Loss: 0.150101 	Validation Loss: 0.145196
Validation loss decreased (0.158643 --> 0.145196).  Saving model ...
Epoch: 8 	Training Loss: 0.134899 	Validation Loss: 0.134358
Validation loss decreased (0.145196 --> 0.134358).  Saving model ...
Epoch: 9 	Training Loss: 0.121010 	Validation Loss: 0.124682
Validation loss decreased (0.134358 --> 0.124682).  Saving model ...
Epoch: 10 	Training Loss: 0.111160 	Validation Loss: 0.114734
Validation loss decreased (0.124682 --> 0.114734).  Saving model ...
Epoch: 11 	Training Loss: 0.101966 	Validation Loss: 0.106838
Validation loss decreased (0.114734 --> 0.106838).  Saving model ...
Epoch: 12 	Training Loss: 0.093625 	Validation Loss: 0.102060
Validation loss decreased (0.106838 --> 0.102060).  Saving model ...
Epoch: 13 	Training Loss: 0.087601 	Validation Loss: 0.095972
Validation loss decreased (0.102060 --> 0.095972).  Saving model ...
Epoch: 14 	Training Loss: 0.081680 	Validation Loss: 0.091928
Validation loss decreased (0.095972 --> 0.091928).  Saving model ...
Epoch: 15 	Training Loss: 0.075197 	Validation Loss: 0.092352
Epoch: 16 	Training Loss: 0.070292 	Validation Loss: 0.087598
Validation loss decreased (0.091928 --> 0.087598).  Saving model ...
Epoch: 17 	Training Loss: 0.065115 	Validation Loss: 0.084549
Validation loss decreased (0.087598 --> 0.084549).  Saving model ...
Epoch: 18 	Training Loss: 0.061174 	Validation Loss: 0.082459
Validation loss decreased (0.084549 --> 0.082459).  Saving model ...
Epoch: 19 	Training Loss: 0.058914 	Validation Loss: 0.079546
Validation loss decreased (0.082459 --> 0.079546).  Saving model ...
Epoch: 20 	Training Loss: 0.054121 	Validation Loss: 0.078417
Validation loss decreased (0.079546 --> 0.078417).  Saving model ...
Epoch: 21 	Training Loss: 0.050982 	Validation Loss: 0.077391
Validation loss decreased (0.078417 --> 0.077391).  Saving model ...
Epoch: 22 	Training Loss: 0.049037 	Validation Loss: 0.074747
Validation loss decreased (0.077391 --> 0.074747).  Saving model ...
Epoch: 23 	Training Loss: 0.045802 	Validation Loss: 0.072876
Validation loss decreased (0.074747 --> 0.072876).  Saving model ...
Epoch: 24 	Training Loss: 0.043257 	Validation Loss: 0.071768
Validation loss decreased (0.072876 --> 0.071768).  Saving model ...
Epoch: 25 	Training Loss: 0.040821 	Validation Loss: 0.071130
Validation loss decreased (0.071768 --> 0.071130).  Saving model ...
Epoch: 26 	Training Loss: 0.038658 	Validation Loss: 0.071208
Epoch: 27 	Training Loss: 0.036397 	Validation Loss: 0.069979
Validation loss decreased (0.071130 --> 0.069979).  Saving model ...
Epoch: 28 	Training Loss: 0.034849 	Validation Loss: 0.069121
Validation loss decreased (0.069979 --> 0.069121).  Saving model ...
Epoch: 29 	Training Loss: 0.034146 	Validation Loss: 0.069191
Epoch: 30 	Training Loss: 0.031644 	Validation Loss: 0.069298
Epoch: 31 	Training Loss: 0.029734 	Validation Loss: 0.070360
Epoch: 32 	Training Loss: 0.029346 	Validation Loss: 0.067803
Validation loss decreased (0.069121 --> 0.067803).  Saving model ...
Epoch: 33 	Training Loss: 0.027256 	Validation Loss: 0.067296
Validation loss decreased (0.067803 --> 0.067296).  Saving model ...
Epoch: 34 	Training Loss: 0.026658 	Validation Loss: 0.066132
Validation loss decreased (0.067296 --> 0.066132).  Saving model ...
Epoch: 35 	Training Loss: 0.025363 	Validation Loss: 0.065438
Validation loss decreased (0.066132 --> 0.065438).  Saving model ...
Epoch: 36 	Training Loss: 0.024144 	Validation Loss: 0.065423
Validation loss decreased (0.065438 --> 0.065423).  Saving model ...
Epoch: 37 	Training Loss: 0.023242 	Validation Loss: 0.066999
Epoch: 38 	Training Loss: 0.021875 	Validation Loss: 0.066241
Epoch: 39 	Training Loss: 0.020930 	Validation Loss: 0.066511
Epoch: 40 	Training Loss: 0.020002 	Validation Loss: 0.065734
Epoch: 41 	Training Loss: 0.019255 	Validation Loss: 0.065650
Epoch: 42 	Training Loss: 0.018258 	Validation Loss: 0.066552
Epoch: 43 	Training Loss: 0.017276 	Validation Loss: 0.065481
Epoch: 44 	Training Loss: 0.016865 	Validation Loss: 0.064185
Validation loss decreased (0.065423 --> 0.064185).  Saving model ...
Epoch: 45 	Training Loss: 0.017011 	Validation Loss: 0.064898
Epoch: 46 	Training Loss: 0.016849 	Validation Loss: 0.065101
Epoch: 47 	Training Loss: 0.015056 	Validation Loss: 0.066197
Epoch: 48 	Training Loss: 0.014886 	Validation Loss: 0.065055
Epoch: 49 	Training Loss: 0.013627 	Validation Loss: 0.064542
Epoch: 50 	Training Loss: 0.013894 	Validation Loss: 0.065223
```

Test the Trained Network

Finally, we test our best model on previously unseen **test data** and evaluate it's performance. Testing on unseen data is a good way to check that our model generalizes well. It may also be useful to be granular in this analysis and take a look at how this model performs on each class as well as looking at its overall loss and accuracy.

`model.eval()`

`model.eval(`) will set all the layers in your model to evaluation mode. This affects layers like dropout layers that turn "off" nodes during training with some probability, but should allow every node to be "on" for evaluation!

In [37]:
```
# initialize lists to monitor test loss and accuracy
test_loss = 0.0
class_correct = list(0. for i in range(10))
class_total = list(0. for i in range(10))

model.eval() # prep model for training

for data, target in test_loader:
    # forward pass: compute predicted outputs by passing inputs to the model
    output = model(data)
    # calculate the loss
    loss = criterion(output, target)
    # update test loss
    test_loss += loss.item()*data.size(0)
    # convert output probabilities to predicted class
    _, pred = torch.max(output, 1)
    # compare predictions to true label
    correct = np.squeeze(pred.eq(target.data.view_as(pred)))
    # calculate test accuracy for each object class
    for i in range(batch_size):
        label = target.data[i]
        class_correct[label] += correct[i].item()
        class_total[label] += 1

# calculate and print avg test loss
test_loss = test_loss/len(test_loader.dataset)
print('Test Loss: {:.6f}\n'.format(test_loss))

for i in range(10):
    if class_total[i] > 0:
        print('Test Accuracy of %5s: %2d%% (%2d/%2d)' % (
            str(i), 100 * class_correct[i] / class_total[i],
            np.sum(class_correct[i]), np.sum(class_total[i])))
    else:
        print('Test Accuracy of %5s: N/A (no training examples)' % (classes[i]))

print('\nTest Accuracy (Overall): %2d%% (%2d/%2d)' % (
    100. * np.sum(class_correct) / np.sum(class_total),
    np.sum(class_correct), np.sum(class_total)))
```

```
Test Loss: 0.059292

Test Accuracy of     0: 99% (973/980)
Test Accuracy of     1: 99% (1126/1135)
Test Accuracy of     2: 98% (1013/1032)
Test Accuracy of     3: 98% (998/1010)
Test Accuracy of     4: 98% (964/982)
Test Accuracy of     5: 97% (867/892)
Test Accuracy of     6: 98% (941/958)
Test Accuracy of     7: 98% (1009/1028)
Test Accuracy of     8: 98% (956/974)
Test Accuracy of     9: 97% (980/1009)

Test Accuracy (Overall): 98% (9827/10000)
```

Visualize Sample Test Results

This cell displays test images and their labels in this format: `predicted (ground-truth)`. The text will be green for accurately classified examples and red for incorrect predictions.

In [38]:
```
# obtain one batch of test images
dataiter = iter(test_loader)
images, labels = next(dataiter)

# get sample outputs
output = model(images)
# convert output probabilities to predicted class
_, preds = torch.max(output, 1)
# prep images for display
images = images.numpy()

# plot the images in the batch, along with predicted and true labels
fig = plt.figure(figsize=(25, 4))
for idx in np.arange(20):
    ax = fig.add_subplot(2, 20//2, idx+1, xticks=[], yticks=[])
    ax.imshow(np.squeeze(images[idx]), cmap='gray')
    ax.set_title("{} ({})".format(str(preds[idx].item()), str(labels[idx].item())),
                 color=("green" if preds[idx]==labels[idx] else "red"))
```